

Signals and Systems

Convolution GUI Project Manual

1. Problem Statement

Design and implement a graphical user interface (GUI) application that allows users to visualize the convolution operation between two signals, an input signal and an impulse response. Your application must support both discrete-time and continuous-time signal modes. The GUI should allow users to:

- Select the signal type - i) discrete or, ii) continuous.
- Enter or select both input and impulse response signals.
- *For discrete-time:* Users should enter signal vectors and starting indices.
- *For continuous-time:* Users should select from standard signal types (impulse, step, triangular, rectangular, sawtooth) and specify parameters (start time, end time, amplitude).
- Compute and animate the convolution process.
- Display three plots: the fixed input signal, the mirrored and shifting impulse response, and the real-time buildup of the output (convolution) signal.

The project must include input validation, user-friendly controls (including animation speed and reset), and clear visual feedback. The solution should be implemented using MATLAB or Python (with a GUI framework and), and be well-documented for future users.

2. Objective

You will build an interactive GUI that visually demonstrates the process of convolution between an input signal and an impulse response. This project will:

- Reinforce your theoretical understanding of convolution in both discrete-time and continuous-time domains.
- Give hands-on experience with computational tools (MATLAB or Python).

- Help you understand how convolution is applied in real-world signal processing systems.

3. Features and Requirements

Your GUI must support the following:

A. Mode Selection

- **Discrete-Time:**
 - Enter: Input Vector, Start Index
 - Enter: Impulse Vector, Start Index
- **Continuous-Time:**
 - For both input and impulse response, select:
 - Signal Type (Impulse, Step, Triangular Pulse, Rectangular Pulse, Sawtooth Pulse)
 - Start Time (t_{start})
 - End Time (t_{end})
 - Amplitude

B. Plots

- Subplot 1: Input Signal
- Subplot 2: Shifting (mirrored) Impulse Response - In real time
- Subplot 3: Output Convolution (animated) - In real time

C. Controls

- Compute Convolution (start animation)
- Reset (clear all)

4. Step-by-Step Operation Guide

- **Launch the App**
- **Select Mode**
- Choose **Discrete-Time** or **Continuous-Time** using the radio buttons.

Convolution Sum Generator

Select Signal Domain

☒ Discrete-Time (DT) ☐ Continuous-Time (CT)

Input Signal

☐ Manual Input ☒ Predefined Signal

Pulse 

Start Time:

Start Time:  

End Time:  

Amplitude:

Impulse Response

☐ Manual Input ☒ Predefined Signal

Pulse 

Start Time:

Start Time:  

End Time:  

Amplitude:

Animation Speed: 

Compute Convolution

Figure: Expected GUI for the project.

- **Input Signal Specification**

- **If Discrete-Time:** Enter the input vector (e.g., [1 2 1 0]) and starting index (e.g., -1)
- **If Continuous-Time:** Select the type of signal from the dropdown, and enter the t_{start} , t_{end} , and amplitude.

- **Impulse Response Specification**

- **If Discrete-Time:** Enter the impulse response vector and starting index.
- **If Continuous-Time:** Select the signal type, t_{start} , t_{end} , and amplitude for the impulse response.

- Press **Reset** to clear all plots and fields if you want to start over.

- **Compute Convolution**

Press the **Compute Convolution** button.

Watch as:

- Subplot 1 displays the stationary input signal.
- Subplot 2 animates the shifting/mirrored impulse response.
- Subplot 3 shows the output convolution being constructed in real time.

5. Extra features for 10% bonus marks

If you want to achieve bonus marks, you can add a few extra features to your project.

- **Set Animation Controls**

- Adjust the speed slider for faster or slower animation as per your preference.
- Press **Reset** to clear all plots and fields if you want to start over.

- **Compute Correlation**

As you will provide two signals as input to the system, you can add an extra button on your GUI, “Compute Correlation”. Pressing on that button will show the correlation between the two provided input.

6. Submission Requirements

- Complete source code (.mlapp for MATLAB or .py for Python).
- Video File Showing how your project is working.
- A brief report (PDF) that includes:
 - Introduction
 - Brief Summary of the Problem Statement
 - Problem Review : in this section you must mention the knowledge required to solve this problem.
 - Methodology: In this section, you have to explain how you approached this problem.
 - Design : Mention how you have designed the GUI to solve the problem.
 - Tool Used : Mention the tool you have used in this project, how the project has improved your skill in this tool.
 - Team Work : Mention how you have worked as a team, You must show a chart mentioning contribution of each team member.
 - Timeline: Provide a timeline of your project, starting from the project announcement date to the project presentation date.

Also, you have to mention if you have added extra features for bonus mark in your project.

7. Justification as a Complex Engineering Problem (CEP)

1. Requires in-depth technical knowledge (PO1, PO2)

Student must understand and implement convolution in both time domains, requiring advanced mathematical and programming skills.

2. Has no straightforward solution—requires design (PO3, PO5)

Designing a GUI, handling both discrete and continuous signals, and visualizing dynamic processes are open-ended design tasks.

Students must develop logic for both numeric convolution and dynamic animation.

3. Involves the integration of multiple concepts (PO9)

Combines signals/systems theory, programming, user interface design, and real-time visualization.

Students must document, present, and possibly collaborate on aspects of the project.

4. Requires use of modern tools and techniques (PO5, PO9)

Students must use contemporary computational tools (MATLAB/Python) and modern UI techniques.

5. Communication of results (PO10)

Requires clear documentation, code commenting, and reporting, making results understandable for both technical and non-technical audiences.

PO Mapping Table

PO No.	Program Outcomes (POs)
	Upon graduation, from the Bachelor of Science in Electrical and Electronic Engineering program, the students are expected to have the ability to:
PO1	Engineering knowledge: Apply knowledge of mathematics, natural science, engineering fundamentals and an engineering specialization to the solution of complex electrical and electronic engineering problems.
PO2	Problem analysis: Identify, formulate, research literature and analyse complex electrical and electronic engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences and engineering sciences.
PO3	Design/development of solutions: Design solutions for complex electrical and electronic engineering problems and design systems, components or processes that meet specified needs with appropriate consideration for public health and safety, cultural, societal, and environmental considerations.

PO5	Modern tool usage: Create, select and apply appropriate techniques, resources, and modern engineering and IT tools, including prediction and modelling, to complex electrical and electronic engineering problems, with an understanding of the limitations.
PO9	Individual work and teamwork: Function effectively as an individual, and as a member or leader in diverse teams and in multi-disciplinary settings.
PO10	Communication: Communicate effectively on complex engineering activities with the engineering community and with society at large, such as being able to comprehend and write effective reports and design documentation, make effective presentations, and give and receive clear instructions.

CEP Criterion	Project Alignment	Mapped PO(s)
Depth of technical knowledge	Signal processing theory, convolution (DT & CT), computational mathematics	PO1, PO2
Unstructured problem solving	No unique solution; design and implementation decisions required	PO2, PO3
System design and integration	GUI, algorithm, and visualization integration	PO3, PO5
Use of modern engineering tools	MATLAB App Designer or Python GUI + Matplotlib	PO5
Integration of engineering principles	Signals, programming, visualization, UX	PO1, PO3, PO5
Communication	Code comments, user documentation, project report	PO10
Teamwork and management (if in groups)	Collaboration, task management, group integration	PO9

8. Assessment Rubric

Criteria & Descriptions	Weight	Related POs	Level (Bloom's)
1. Correctness of Convolution Implementation (DT & CT)- Accurate calculation, proper handling of limits, correctness for various signal types (discrete & continuous)	30%	PO1, PO2	C3–C4
2. System & GUI Design- User-friendliness, logical input flows, clear labeling, proper mode selection, error prevention, reset, input validation	20%	PO3, PO5	C5
3. Visualization & Animation- Animation of convolution, clarity of subplots, real-time feedback, smoothness and educational value of visualization	15%	PO3, PO5	C5
4. Input Validation, Error Handling, Edge Cases- Robust handling of wrong or edge inputs, stability, warnings/messages for user errors	10%	PO2	C4
5. Documentation and Code Comments- Modular, readable code, sufficient inline comments, use of functions, code structure	10%	PO5, PO9, PO10	C5
6. Report & Communication- Quality and clarity of submitted report, screenshots, sample runs, discussion of results, reflection on learning	10%	PO10	C6
7. Teamwork and Initiative (if group project)- Equitable participation, collaboration, distribution of work, creativity, and innovation	5%	PO9	C5

**If you have any questions, contact the course instructor as early as possible.
Good luck!**